

DAWN Access-Domain Schema — Design Rationale

For: CTO / engineering review Audience: someone who knows Solana at a high level, and runs MPSK BSS at production scale. Scope: the lean access-domain program as deployed to devnet at `rHSumT63fgwAsHhKbR39AijjY28H99VuvM8xNaNehkj` (program v2.1.0).

1. What this document is

When we narrowed the protocol to a production-MPSK shape, we made a sequence of schema decisions that aren't immediately obvious from the code. This document captures **what we chose, what we considered, and why we landed where we did**, so the architectural decisions don't have to be re-discovered the next time someone reads the program.

The constraints that drove every choice:

1. **Key custody is the primary access-control mechanism.** No tokens, no economic gating. If you hold the right key, you can sign the right tx.
 2. **Operator runs many BSS sites.** Schema must scale to N AccessDomains per operator, M customers per AccessDomain.
 3. **Cold-admin keys must stay cold.** Day-to-day operations (customer enrollment, band changes) cannot require the operator's HSM signature.
 4. **The decryption authority is a separate process from the cold admin.** A daemon running on the dawn node holds the secret that recovers PSKs; the operator's HSM does not.
 5. **Off-chain source-of-truth (Nautobot, BSS database) must reconcile to on-chain records 1:1.**
-

2. The three-key model

Every production deployment has three on-chain identities and one off-chain process:

Role	Key location	Signs	Frequency
Cold admin	Operator HSM / cold storage	<code>initialize_config</code> , <code>update_config_authority</code> , <code>add_access_domain</code> , <code>register_auth_method</code> , <code>grant/revoke_domain_authority</code> , direct credential cleanup	Setup events; ops emergencies
Registrar	Operator-api hot wallet	<code>register_credential_for</code> (customer enrollment)	Continuous, per signup
Config plane manager	Ops wallet	<code>update_auth_method_params</code> (e.g. drop a band, rotate WPA std)	Occasional ops actions
Control plane node (<i>off-chain</i>)	Dawn node Ed25519 secret	Never signs txs	Decrypts every credential

The schema is structured around making this separation enforceable: each role only has the on-chain authority it needs, and the cold key is required for **policy** changes, not **operations**.

3. The account graph

```
Config (singleton, cold-admin authority)
├── (gates: who can register DeviceModel)
│   DeviceModel (one per (type, mfg, model); shared across operators)
│       └── Device (one per physical box; owner = whoever owns the appliance)
│           └── ┌── pointed at by ───┐
│               │                   │
├── AccessDomain (one per BSS / SSID; owner = cold operator)
│   ├── control_plane_device → Device whose owner decrypts credentials
│   ├── gateway_device      → optional Device (data-plane gateway)
│   ├── external_uuid       → optional Nautobot site UUID for SoT reconciliation
│   └── ┌── AuthMethod (one per AccessDomain – keyed on method_type byte)
│       │   ├── parameters[256] (PSKMethodParams: tri-band SSID + WPA std + rotation)
│       └── Credential[] (one per (AccessDomain, AuthMethod, customer) tuple)
│           ├── sealed_payload[128] (libsodium sealed-box → control_plane_device.owner)
│       └── DomainAuthority[] (Tier-2 role grants on this AccessDomain)
│           ├── role = Registrar → may sign register_credential_for
│           └── role = ConfigPlaneManager → may sign update_auth_method_params
```

Read this top-to-bottom: each level exists to enable the level below it. The shape is **deliberately minimal** — every account here was either required by the production model or kept for forward compatibility with documented future work.

4. Schema decisions, justified

4.1 `AccessDomain` is a top-level account

Decision: `AccessDomain` is keyed on `(owner, sha256(name))`, not nested under any other account.

Alternative considered: Nest under `LocalDomain` (the management-plane abstraction we have for other reasons).

Why we chose top-level: an SSID is a first-class network identity — operators think of "the cafe wifi" as the unit of management, not as a child of a management-plane construct. Putting it at the top means:

- Operators discover their ADs with one `getProgramAccounts` filter (`owner == operator_pubkey`)
- The PDA seed is stable even if mgmt-plane structure changes later
- The bridge's discovery filter is simple

The hash-of-name in seeds: PDA seeds have a fixed-byte budget. Hashing the variable-length SSID gives us a predictable 32-byte input regardless of SSID length, while keeping the on-chain `name: String` field human-readable.

4.2 `AccessDomain.control_plane_device` is required

Decision: Every `AccessDomain` MUST point at a `Device` PDA whose owner decrypts credentials. This is non-optional.

Alternative considered: Embed the decryption pubkey directly on the `AccessDomain` (`decrypt_pubkey: Pubkey`).

Why we chose Device indirection:

- Operators often run a redundant pair of AAA appliances. By naming a `Device` PDA (which has its own metadata: model, location, MAC), we get on-chain inventory of the AAA hardware along with a clean way to **re-point** an `AccessDomain` at a different appliance with `set_control_plane_device` rather than encoding the wallet directly.
- The `Device` PDA has stable identity (owner + model + name + MAC) that survives wallet rotations. If the operator wants to migrate the AAA secret to a new wallet, they can — by adding a new `Device` for the new wallet and pointing the `AccessDomain` at it.
- Decryption authority is "the wallet that owns this `Device` account," which is a one-line on-chain lookup.

The PDA itself is unsignable — it's an off-curve point. The wallet pubkey *recorded as* `Device.owner` is what holds the Ed25519 secret and decrypts. This is the only place this distinction matters in our schema, and it's worth flagging because some bridge implementations get confused by it.

4.3 AuthMethod keyed on (access_domain, method_type)

Decision: One AuthMethod per (AccessDomain, method_type byte). PSKMethodParams are stored in a 256-byte fixed buffer, mutable in place.

Alternatives considered:

- Key on (access_domain, hash(parameters)) — every parameter change creates a new account.
- Multiple AuthMethods of the same type per AccessDomain — for evolving WPA configurations.

Why we chose mutable-in-place:

- An MPSK BSS conceptually has *one* security profile. Rotating the WPA standard or dropping a band shouldn't force closing every customer credential.
- A new AuthMethod account per param change would either invalidate every Credential (since they reference auth_method PDA) or require complex migration logic.
- In-place updates require a separate authority (ConfigPlaneManager) so the cold admin doesn't need to sign for routine band changes.

The 256-byte buffer: PSKMethodParams is 128 bytes today. The trailing 128 bytes are reserved zero — room for per-band fields (band-specific encryption, MFP requirements, etc.) without an account-size migration. We sized the buffer for forward-fit rather than minimum.

4.4 PSKMethodParams carries multi-band SSIDs

Decision: Three SSID slots in the same params struct (2.4 / 5 / 6 GHz), each with a length byte + 32-byte label slot. At least one must be active.

Alternatives considered:

- One SSID per AuthMethod, three AuthMethods per AccessDomain (one per band).
- Variable-length array of SSIDs.

Why we chose three fixed slots:

- Production BSSes commonly broadcast the *same* SSID across multiple bands for transparent steering. Modeling three independent AuthMethods would require correlating them off-chain anyway.
- A variable-length array would push us into Borsh-encoded Vec<> semantics inside the params buffer, which is awkward for in-place mutation.

- Three fixed slots covers 2.4/5/6 GHz comprehensively. If 7 GHz becomes standard later we burn a backward-compat fork.

Length byte = 0 means "band inactive": dropping a band is just `update_auth_method_params` with that slot's length byte zeroed. The band's label slot bytes are zeroed alongside. The bridge re-fetches and observes which bands are now empty.

4.5 **Credential identity is (access_domain, auth_method, customer)**

Decision: Credential PDA is seeded on three pubkeys, even though `auth_method` is currently always derivable from `access_domain` (one method per AD).

Why include `auth_method` in the seeds:

- If we ever support multiple co-existing methods on the same AccessDomain (PSK + EAP, for instance), the existing PDAs don't collide.
- It's belt-and-suspenders today, but the cost is negligible (32 extra bytes in the PDA derivation seed) and removing it later would be a hard schema break.

customer here is a bare wallet pubkey, not a Device. A credential is per-person, not per-device. The customer's device(s) authenticate against the access point using the PSK; the on-chain identity is the customer's signing key. This decouples PSK identity from device identity, which matches how MPSK actually works (one PSK can be used by multiple of the customer's devices, plus the customer can roll devices without re-enrolling).

4.6 **Credential.sealed_payload[128] is opaque on-chain**

Decision: PSKs are encrypted off-chain to the `control_plane_device.owner` pubkey using libsodium's sealed-box. The on-chain program treats the 128-byte buffer as opaque — never reads, decodes, or validates it.

Wire layout: 32 bytes ephemeral X25519 pubkey, then 96 bytes of (ciphertext + Poly1305 tag). The plaintext is a fixed-size 80-byte frame regardless of PSK length, so PSK length isn't leaked through ciphertext length.

Why opaque + fixed-size:

- Anyone reading the chain sees ciphertext only. Compromising the operator's HSM or the registrar's hot wallet doesn't reveal customer PSKs.
- Public auditability is preserved — a regulator can verify the protocol's behavior without seeing PSKs.

- The plaintext is versioned (a 1-byte version + 1-byte kind + 1-byte length header) so future credential types (EAP keys, IPsec material) can use the same envelope.
- 80 bytes covers the WPA PSK limit (63 utf-8 bytes) with headroom; libsodium overhead (48 bytes) is fixed.

Operator-api implementation requirement: sealed-box envelopes are non-deterministic (fresh ephemeral keypair per seal). For idempotent retry, the operator-api must persist the envelope bytes locally **before** submitting the tx, and replay the same bytes on retry. We documented this and built a helper (`registerCredentialForIdempotent`) that byte-compares on-chain state against the operator-api's local copy.

4.7 `DomainAuthority` is a separate Tier-2 grant account

Decision: Delegated roles (Registrar, ConfigPlaneManager) live in their own `DomainAuthority` accounts, one per (domain, role, authority) tuple.

Alternatives considered:

- A `Vec<Pubkey>` of registrars on `AccessDomain` itself.
- Embed two pubkey fields directly: `registrar: Pubkey` , `config_plane_manager: Pubkey` .

Why we chose separate accounts:

- **Rotation is a frequent operation.** Hot operator-api wallets get rotated for security hygiene, breach response, version upgrades. Closing one `DomainAuthority` and creating another is two cheap txs and reclaims the old one's rent. Mutating an embedded field on the `AccessDomain` forces every read to dereference and check the freshness.
- **Audit trail is automatic.** Every grant/revoke produces its own tx + event. The operator's transaction history is the rotation audit log.
- **Multiple registrars are possible** — useful for blue/green deploys where you grant v2 *before* revoking v1, then test both, then revoke v1.
- **Per-grant labels and expiries.** Each `DomainAuthority` has its own `label: Option<String>` and `expires_at: Option<i64>` . Lets ops humanly identify a grant ("operator-api-prod-us-east") and gives the program an enforceable expiry without any background sweeper.

Roles are an enum with stable byte values. Adding a role variant only consumes the next free byte — existing PDAs keep their seeds. Roles are part of the PDA seed, so the same authority can hold multiple roles (e.g. one wallet that's both Registrar and ConfigPlaneManager) at distinct PDAs.

Three roles defined; only two used today:

Role byte	Name	Authority for
0	Registrar	Mints / revokes Credentials
1	ConfigPlaneManager	Updates AuthMethod params

Future variants (AuthMethodManager , PlanCreator , AdminDeputy) are reserved in the design but not in code yet — they'll be added when we have a concrete need rather than speculatively.

4.8 Optional Option<Pubkey> subscription / plan on Credential

Decision: Credential has two Option<Pubkey> fields that are **always None** in the current program (subscription and plan).

Why keep dead fields: in earlier iterations the Credential connected to a commercial relationship (subscription with billing terms, plan with QoS contract). The lean access-domain build dropped both, but we kept the fields in the schema:

- Adding fields later requires an account-data resize, which is operationally painful (re-allocation, migration).
- Removing them from the struct would require closing every Credential and re-creating with the smaller layout, which would invalidate every customer.
- Keeping them None costs 2 bytes per Credential (the option tags); the cost is negligible.

This is **forward-compat scaffolding**. When we add a commercial layer later (subscriptions, billing), no schema migration is needed; the program just starts writing them as Some.

4.9 external_uuid on AccessDomain

Decision: AccessDomain has an Option<[u8; 16]> field for binding to off-chain SoT records.

Why on-chain:

- Nautobot (or whatever the operator's BSS uses) is the human-facing source of truth. The on-chain record needs a stable handle to reconcile against.
- Storing the UUID on-chain (rather than off-chain in a side table) means the binding survives operator-side database migrations and stays auditable.
- 16 bytes is exactly RFC 4122 UUID size; we don't need a longer or shorter handle.

It's optional because pure-Solana deployments without an off-chain BSS shouldn't be forced to populate it. In production every AccessDomain we deploy has it set.

4.10 Event attribution: `created_by` / `revoked_by` (v2.1.0)

Decision: Every emit event has a `created_by` or `revoked_by` field carrying the caller pubkey at emit time.

Alternative we initially shipped (v2.0.0): Attribution was implied — for `register_credential_for`, you knew the caller was either the AccessDomain owner (direct mint) or some Registrar. To find which, you fetched the tx and inspected the signers list.

Why we changed it: the SoT-bridge needed to know which Registrar minted each credential for ops audit ("registrar v3 minted these 47 credentials before we rotated to v4"). Per-tx fetching is hot-path-killing for a streaming bridge. Adding 32 bytes to four event types removes the dependency cleanly.

Disambiguation pattern:

```
if event.created_by == accessDomain.owner:
    → direct mint, no Registrar grant invoked
else:
    → mint via Registrar; the Registrar's authority pubkey IS event.created_by
```

The bridge maintains a map of `{ authority_pubkey → DomainAuthority PDA }` from `DomainAuthorityGranted` events and looks up which Registrar was active when each credential was minted. No per-tx RPC calls.

5. What we deliberately did NOT do

For completeness — choices we considered and rejected:

5.1 No on-chain RADIUS state

Why not: the AAA daemon is the authoritative copy of "who is currently allowed on the network." Putting a mirror of that on-chain would force every association event to produce a tx — unworkable at the rate phones associate/reassociate. The chain holds the **policy** (who is granted access, with what PSK); the AAA daemon holds the **state** (who is currently associated).

5.2 No on-chain credential rotation tracking

Why not: when a customer's PSK is rotated, the operator-api closes the old Credential PDA and creates a new one at the same PDA with the new sealed_payload. There's no "rotation count" or "previous_psk_hashed" field. The tx history is the rotation log; the on-chain account is current state only.

5.3 No Connection accounts

Why not: earlier iterations had a Connection (entity_a, entity_b, auth_method) for site-to-site IPsec or peer authentication. Production MPSK doesn't need this — every connection in MPSK is implicitly client → AP, with no on-chain peer-to-peer relationship. We removed Connection entirely from the lean build.

5.4 No tokens, fees, or plans

Why not: the access-domain plane is for permissioned BSS deployment. Customer billing happens off-chain in the operator's BSS. There's no economic gating, no native token, no plan-tier enforcement — anyone with a valid Registrar grant can mint credentials. Re-introducing the token/fee layer is a separate program build (access-domain-with-billing or similar) that imports the same schema and adds the commercial accounts on top.

5.5 No automatic expiry-driven revocation

Why not: DomainAuthority grants have an expires_at: Option<i64> . The program **rejects** an expired grant when used, but it does NOT automatically close the account when expiry passes. The operator must explicitly revoke_domain_authority_for_access_domain to reclaim rent and clean up.

This is intentional. Solana has no background scheduler; an "auto-revoke at slot X" mechanism would require either:

- A keeper/sweeper running off-chain to call revoke at expiry (operationally complex, single point of failure)
- A background-on-read pattern where every use of the grant pre-expiry succeeds and post-expiry rejects (which is what we have — minus the rent reclaim)

We chose the simpler model and let the operator script revocation as part of their rotation workflow.

6. Operational invariants the schema enforces

These are the contracts the program guarantees:

1. **Only the cold admin can `register_auth_method`, `add_access_domain`, or grant/revoke a `DomainAuthority`.** Hot wallets cannot escalate.
2. **Only an active Registrar grant can mint credentials in the registrar-mediated path.** Expired grants are rejected on use.
3. **Only the credential's authority OR the AccessDomain owner can revoke a credential.** Customers can self-revoke; operators can clean up. Registrars cannot revoke (they can only mint).
4. **Credentials are unique per `(access_domain, auth_method, customer)` tuple.** Two simultaneous credentials for the same customer at the same SSID is impossible by PDA constraint.
5. **AuthMethod parameters are mutable in place by the AccessDomain owner OR a live `ConfigPlaneManager`.** The cold admin and the ops wallet both have authority; neither can mutate without a signature.
6. **`sealed_payload` is opaque to the program.** No on-chain validation of plaintext format, length, or content — that's the operator-api's responsibility to seal correctly and the bridge's to decrypt correctly.
7. **All emit events carry `created_by` / `revoked_by` (v2.1.0+) for clean attribution.** Bridge implementations can reconstruct the full audit trail without per-tx RPC.

7. Quick numbers

Property	Value
Program binary size	533 KB
Number of instructions	16
Number of account types	9
Number of event types	16
Number of error codes	44
Mainnet deploy rent	~3.71 SOL one-time
Per-customer Credential PDA rent	~0.0028 SOL
Per-AccessDomain rent	~0.0027 SOL
Per-Registrar grant rent	~0.0019 SOL
Anchor framework	0.31.1
Solana runtime	2.1.x

At current SOL prices (~~\$150~~), ~~per-customer cost is~~ ****\$0.42**** in rent, fully reclaimable on revocation. A 10,000-customer BSS site has ~\$4,200 of rent locked across all credentials, each individually recoverable.

8. Open work

Items intentionally deferred, in priority order:

1. **Audit.** Slim 16-instruction surface, no token interactions, no CPI to other programs except SystemProgram for account creation. Estimated audit budget \$15–30K range with a top-tier Solana auditor (Halborn / Neodyme / OtterSec). Pre-requisite for mainnet.
2. **Mainnet deploy.** Same artifacts as devnet; just needs ~7.4 SOL of mainnet SOL on the deployer wallet and an audit-clean program build.

3. **Operator-api integration doc.** The SoT-bridge has a self-contained protocol doc (`docs/sot-bridge-protocol.md`). The operator-api side (Python BSS backend) needs an equivalent: WAL pattern for sealed-box idempotency, registrar rotation playbook, expiry hygiene.
4. **Multi-bridge filter pattern (Model B).** Currently the bridge filters AccessDomains by `owner == operator_pubkey` . For multi-bridge deployments the more correct filter is `control_plane_device == bridge_device_pda` (each bridge sees only ADs that point at it). One-line change in the bridge; documented in the bridge protocol doc but not yet rolled out.
5. **Reconciliation loop on the bridge.** Currently the bridge replays history at startup + streams live. A periodic full-rebuild (every N minutes) catches anything missed during WS drops. Hygiene; not blocking.

Last updated: program v2.1.0, slot 461303727, devnet
`rHSumT63fgwAsHhKbR39AijjY28H99VuvM8xNaNehkj` .